Cahier des Charges : Projet Vigor

Assistant de nutrition prédictive et de santé globale optimisé par l'IA

---

1. Présentation du projet

Vigor est une application mobile de santé qui utilise l'Intelligence Artificielle pour transformer le suivi de l'alimentation. Contrairement aux simples compteurs de calories, Vigor analyse les corrélations entre la **nutrition**, le **sommeil**, le **stress**, l'**activité physique** et l'**humeur**.

L'application reconnaît les plats de la **cuisine marocaine** grâce à la vision par ordinateur (YOLO), exploite les capteurs du smartphone et des objets connectés Bluetooth, et propose un **Coach IA** basé sur une architecture RAG pour un accompagnement personnalisé (perte de poids, prise de masse, régime diabétique, mode étudiant, etc.).

2. Contexte et problématique

- Les applications existantes (calories, macros) reconnaissent mal les plats marocains (tajine, couscous, harira, pastilla, rfissa, etc.).
- Elles traitent l'alimentation isolément, sans lien avec le sommeil, le stress ou l'activité.
- Elles ne tiennent pas compte du budget (étudiants), de la religion (halal, Ramadan) ni des contraintes locales.
- Elles fonctionnent mal sans connexion Internet.

3. Objectifs

Objectifs généraux

- **Santé et bien-être :** offrir une vision holistique en liant l'assiette à l'état physique et émotionnel.
- **Accessibilité locale :** valoriser la gastronomie marocaine grâce à un modèle entraîné sur un dataset spécifique.
- **Inclusion économique :** proposer un mode « Étudiant » pour manger sainement avec un budget restreint (ex. moins de 25 MAD par jour ou par repas).
- **Technologique :** déployer des modèles de vision (YOLO, MediaPipe) et de langage (LLM + RAG) dans une architecture cloud/mobile fluide.

Objectifs mesurables

- Précision de détection supérieure à 85 % sur les plats marocains courants (classes retenues).
- Temps de traitement d'une image inférieur à 3 secondes.
- Fonctionnement des fonctions de base (scan simple, journal, statistiques) sans connexion.

4. Acteurs du système

Acteur	Rôle
Utilisateur final	Scanne ses repas, saisit ses données (eau, sommeil, humeur), discute avec le coach, suit ses statistiques, participe aux défis
Coach IA (agent virtuel)	Analyse les données de l'utilisateur, répond aux questions via RAG, envoie des conseils proactifs
Administrateur	Gère la base nutritionnelle, valide les corrections de scans, gère les documents de la base de connaissances et les partenariats (salles de sport), analyse les tendances globales
Objets connectés (externes)	Bracelet cardiaque BLE, balance connectée (optionnel), fournissant des mesures en temps réel

5. Besoins fonctionnels

5.1 Authentification et profil

- **F1.** Inscription et connexion (email / mot de passe) via Supabase Auth, avec persistance de session.
- **F2.** Profil utilisateur : nom, photo (stockage cloud), âge, taille, poids, objectif (perte de poids, prise de masse, maintien, diabète), allergies, régime (halal, végétarien), budget.
- **F3.** Gestion des préférences : langue (français, arabe/darija, anglais), notifications, mode sombre.

5.2 Cœur IA et vision (caméra)

- **F4. Scanner alimentaire :** photo d'un repas, détection automatique des aliments (plats marocains, fast-food, boissons) avec score de confiance, et estimation des calories et macronutriments.
- **F5. Correction par l'utilisateur :** l'utilisateur peut modifier ou ajouter un aliment détecté et ajuster les portions. Les corrections sont enregistrées pour améliorer le dataset.
- **F6. Analyse vidéo :** courte vidéo d'une table complète. Des images sont extraites à intervalles réguliers, la détection est appliquée, puis les doublons sont fusionnés pour détecter plusieurs portions.
- **F7. Scan de code-barres :** identification des produits emballés et de leurs valeurs nutritionnelles.
- **F8. Analyse par MediaPipe :** comptage automatique de répétitions d'exercices (squats, pompes) via la détection de pose, enregistré comme activité physique. *(Variante possible : estimation de la taille de portion avec la main comme référence.)*

5.3 Microphone et commande vocale

- **F9. Saisie vocale des repas :** l'utilisateur dicte son repas en français ou darija (« j'ai mangé un tajine et deux verres de thé »). La transcription est analysée par le LLM pour extraire les aliments et les quantités.
- **F10. Chat vocal avec le coach :** question orale, réponse textuelle (et vocale en option).

5.4 Coach conversationnel (RAG)

- **F11.** Chat interactif pour des conseils en temps réel (ex. « Quoi manger après ma séance de sport ? »).
- **F12. Base de connaissances RAG :** guides de nutrition, recommandations pour le diabète, nutrition sportive, recettes marocaines équilibrées. Les documents sont découpés, vectorisés et stockés (pgvector).
- **F13.** Les réponses combinent : le profil de l'utilisateur, ses 7 derniers jours de données et les passages pertinents de la base, avec **citation des sources**.
- **F14. Conseils proactifs :** suggestions déclenchées par les données (ex. « Vous avez dormi 5 h, évitez les snacks sucrés cet après-midi »).
- **F15. Garde-fous :** le coach ne pose aucun diagnostic médical. Un avertissement est affiché, et les conseils du mode diabétique restent prudents.

5.5 Suivi holistique et capteurs

- **F16. Journal holistique :** enregistrement de l'eau, du sommeil, de l'activité physique, de l'humeur et du poids.
- **F17. Capteurs du smartphone :** podomètre et détection d'activité (accéléromètre/gyroscope), estimation du sommeil (inactivité nocturne, luminosité ambiante), intégration optionnelle de Health Connect / Apple Health.
- **F18. Bluetooth (BLE) :** appairage d'un bracelet cardiaque pour afficher la fréquence cardiaque en direct pendant l'entraînement, et d'une balance connectée (optionnel). Un module ESP32 peut simuler un capteur pour la démonstration.
- **F19. Historique des mesures :** stockage et visualisation des relevés de capteurs.

5.6 Analytique et personnalisation

- **F20. Analytique avancée :** graphiques de corrélation (ex. « L'impact du manque de sommeil sur vos envies de sucre », humeur vs horaires de repas, hydratation vs énergie).
- **F21. Insight hebdomadaire :** résumé automatique des tendances et prévisions simples (tendance calorique, risque de rupture de série).
- **F22. Génération de menus :** programmes de repas hebdomadaires selon le budget, les allergies, la religion et l'objectif, avec liste de courses et coût estimé en MAD.
- **F23. Mode Étudiant :** menus et recettes à petit budget.
- **F24. Mode Ramadan :** planification iftar/suhoor et hydratation adaptée.

5.7 Localisation et cartes

- **F25. Suggestions locales :** carte des restaurants sains, marchés, salles de sport et lieux à prix étudiant à proximité (OpenStreetMap).
- **F26.** Recettes marocaines équilibrées et filtrage par distance, prix et type de cuisine.
- **F27.** Rappels géolocalisés (ex. à l'approche d'une salle de sport partenaire).

5.8 Engagement et communauté

- **F28. Gamification :** badges, niveaux, séries (streaks).
- **F29. Défis entre amis :** classements en temps réel (Supabase Realtime).
- **F30. Notifications :** rappels d'eau, de repas, de sommeil, résultats de défis (centre de notifications avec onglets Toutes / Non lues / Système).

5.9 Mode hors ligne

- **F31.** Scan basique via modèle TFLite embarqué, journal et statistiques consultables sans réseau.
- **F32.** Synchronisation automatique avec le serveur au retour de la connexion.

5.10 Rapports et export

- **F33.** Génération d'un rapport PDF hebdomadaire (nutrition, sommeil, poids, graphiques) partageable avec un nutritionniste.
- **F34.** Export et suppression des données personnelles à la demande.

5.11 Administration (interface web)

- **F35.** Gestion de la base d'aliments et de plats (valeurs nutritionnelles, portions types).
- **F36.** Revue des scans corrigés par les utilisateurs pour enrichir le dataset.
- **F37.** Gestion des documents de la base RAG (ajout, indexation, suppression).
- **F38.** Tableau de bord de tendances agrégées et anonymisées.

6. Besoins non fonctionnels

Catégorie	Exigence
Précision	Confiance supérieure à 85 % sur les classes de plats marocains courants retenues
Performance	Traitement d'une image par l'IA en moins de 3 secondes ; interface fluide (60 fps visé)
Offline-first	Base locale (Isar ou Hive) garantissant l'accès à l'historique sans réseau, avec synchronisation différée
Confidentialité	Chiffrement des données de santé (en transit et au repos), stockage sécurisé local, consentement explicite, droit d'export et de suppression, respect de la loi marocaine 09-08 et des bonnes pratiques RGPD
Sécurité	Row Level Security (RLS) sur toutes les tables utilisateur, jetons d'authentification, validation des fichiers téléversés, clés API non exposées dans l'application
Fiabilité	Gestion des erreurs réseau, reprise après coupure, journalisation côté serveur
Portabilité	Android en priorité, iOS compatible (Flutter), interface web pour l'administration

Catégorie	Exigence
Utilisabilité	Interface claire, multilingue (français, arabe, anglais), thème sombre, accessibilité de base
Maintenabilité	Architecture par fonctionnalités, code documenté, déploiement conteneurisé (Docker)

7. Architecture technique

Application Flutter (Dart)

├– Isar / Hive (offline) ├– Caméra, micro, capteurs

├– BLE (flutter_blue_plus) └– Cartes (flutter_map + geolocator)

| REST / WebSocket

FastAPI (Python)

├– /scan → YOLOv8 (détection de plats)

├– /coach → LangChain + LLM + RAG

├– /menu → génération de menus

└– /insights → corrélations et statistiques

|

Supabase (PostgreSQL) en Docker

├– Auth ├– Storage ├– Realtime └– pgvector (embeddings RAG)

Couche	Technologie / Choix	Justification technique
Application mobile	Flutter + Riverpod	Réutilisation du socle existant et gestion d'état performante.
Base locale (offline)	Drift (SQLite) ou Hive CE	Isar étant moins maintenu, Drift est idéal pour les données relationnelles et la synchronisation.
Backend & API	FastAPI + Uvicorn	Python natif, parfait pour orchestrer tous les modèles d'IA avec une faible empreinte mémoire.
BDD / Auth / Storage	Supabase Cloud + pgvector	Évite de surcharger les 16 Go de RAM avec des conteneurs Docker locaux.

Couche	Technologie / Choix	Justification technique
Vision serveur	Ultralytics YOLOv8 + ONNX Runtime / OpenVINO	OpenVINO/ONNX exploitera au maximum la puissance de ton CPU Intel Core i7 13th gen pour une inférence rapide sans GPU.
Vision mobile (offline)	Export TFLite + tflite_flutter	Scan hors ligne embarqué directement sur le téléphone.
Pose / Répétitions	MediaPipe (Python) ou google_mlkit_pose_detection	Comptage par angles articulaires très fluide sur CPU.
LLM Local (Coach)	Ollama avec modèle quantifié (qwen3.5:4b ou qwen2.5:7b)	Légèreté et rapidité d'exécution sur 16 Go de RAM, avec API compatible OpenAI.
Orchestration RAG	Code Python natif ou LangChain	Approche légère, facile à documenter et à justifier lors de la soutenance.
Embeddings	multilingual-e5-small ou BAAI BGE-M3	Support multilingue (français/darija).
Voix	faster-whisper (serveur) ou speech_to_text (natif)	Le français sert de repli fiable face aux limites actuelles sur le darija.
Cartographie & BLE	flutter_map (OSM) + flutter_blue_plus	Solutions 100% open source. ESP32 utilisé pour simuler le bracelet.
Annotation Dataset	Label Studio ou CVAT	Outils open source pour labelliser tes plats marocains.
Notifications	Firebase Cloud Messaging (FCM)	

8. Modèle de données (tables principales)

Table	Contenu
profiles	Identité, objectif, allergies, régime, budget

Table	Contenu
foods / dishes	Aliments et plats avec valeurs nutritionnelles et portions types
meals / meal_items	Repas enregistrés et leurs composants
scan_feedback	Corrections utilisateur sur les détections
daily_logs	Eau, sommeil, humeur, poids
activities	Séances et répétitions (MediaPipe, capteurs)
sensor_readings	Mesures BLE et capteurs du téléphone
places	Lieux sains à proximité (coordonnées, prix, type)
documents / document_chunks	Base RAG avec colonne embedding (pgvector)
chat_messages	Historique des conversations avec le coach
menus / menu_items	Programmes de repas générés
badges / user_badges / challenges	Gamification et défis
notifications	Centre de notifications

9. Planning prévisionnel (6 semaines)

Phase	Durée	Tâches principales
1. Conception	1 sem.	Maquettes UI/UX (Figma), modélisation de la base, collecte du dataset et premier entraînement YOLO
2. Infrastructure	1 sem.	Supabase local (Docker), configuration FastAPI, authentification, structure du projet Flutter (réutilisation du socle existant : connexion, profil, navigation)
3. Cœur IA	1 sem.	Scanner (caméra → YOLO → macros), saisie vocale, premier export TFLite
4. Coach et suivi	1 sem.	Chat RAG, journal holistique, génération de menus, interfaces Flutter principales
5. Capteurs, analytique et offline	1 sem.	Cartes, Bluetooth, capteurs, corrélations, base locale et synchronisation
6. Tests et démonstration	1 sem.	Gamification, débogage, optimisation de l'inférence, rapport PDF, préparation de la soutenance

10. Risques et mesures

Risque	Mesure
Dataset de plats marocains insuffisant	Commencer par 15 à 25 classes (100 à 200 images par classe) via sources publiques (Roboflow, etc.) et photos personnelles ; augmentation de données
Estimation des calories imprécise	Utiliser une table de portions types par plat plutôt qu'une régression sur pixels, avec correction manuelle possible
Latence supérieure à 3 s	Modèle YOLOv8n/s, quantification TFLite, redimensionnement des images, inférence embarquée
Hallucinations du LLM	RAG avec citation des sources, garde-fous et avertissement non médical
Indisponibilité du matériel BLE	Simulation par ESP32 ou application nRF Connect
Dérive de planning	Prioriser le MVP (scanner, journal, coach), traiter cartes, BLE et gamification en second

11. Périmètre MVP et évolutions

MVP (indispensable pour la démonstration) : authentification, scanner photo, journal holistique, coach RAG, statistiques de corrélation simples, mode hors ligne de base.

Évolutions : analyse vidéo complète, comptage de répétitions MediaPipe, BLE (bracelet), cartes et suggestions locales, génération de menus, gamification et défis, mode Ramadan, rapport PDF, interface web d'administration.

12. Livrables

- Application mobile Flutter fonctionnelle
- API FastAPI et base Supabase (schéma SQL, politiques RLS)
- Modèle YOLO entraîné et exporté (PyTorch et TFLite) avec rapport de performance
- Base de connaissances RAG indexée
- Documentation (README structuré, schéma d'architecture, guide d'installation)
- Démonstration et support de soutenance